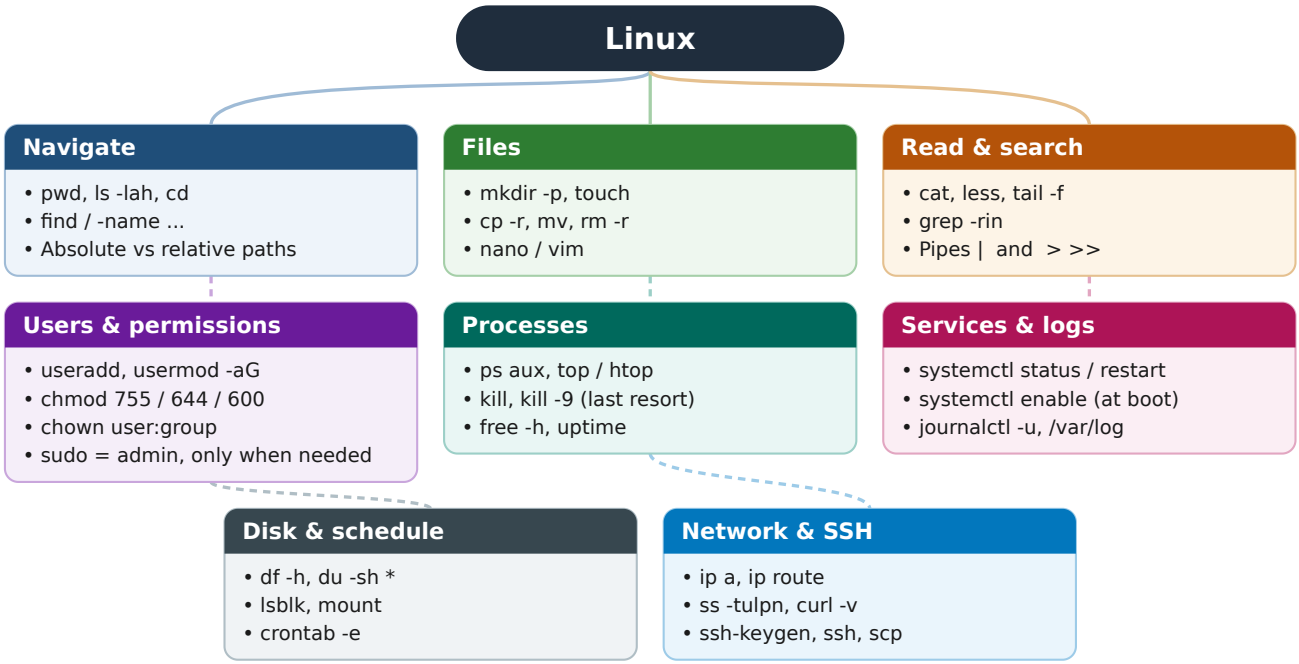
3. Linux

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 3 of 18 · Thirupathi Keloth · v2.0

Contents

3.0 How to use these notes
3.1 What Linux is and where it is used
3.2 Linux architecture and boot process
3.3 The shell and the command line
3.4 The file system
3.5 Working with files and directories
3.6 Viewing files
3.7 Searching
3.8 Text processing
3.9 Text editors
3.10 Redirection, pipes and exit codes
3.11 Users and groups
3.12 Permissions in depth
3.13 Processes and jobs
3.14 System resources and performance
3.15 Package management
3.16 systemd and services
3.17 Logs
3.18 Storage, file systems and LVM
3.19 Archiving, compression and file transfer
3.20 Environment and shell configuration
3.21 Networking on Linux
3.22 SSH in depth
3.23 Scheduling: cron, at and timers
3.24 Time synchronization
3.25 Linux security and hardening
3.26 Hands-on lab: nginx web server and reverse proxy
3.27 Troubleshooting scenarios
3.28 Command reference by task
3.29 Interview questions and answers
3.30 Self-check: I can...



3.0 How to use these notes

- Practise every command in a terminal while you read: WSL2 Ubuntu on Windows, a VirtualBox/VMware VM, or a cloud instance.

- Commands starting with `sudo` change the system; run them only on a lab machine until you are confident.
- `$` at the start of an example line means "run as a normal user"; `#` means "run as root". Don't type the `$` / `#`.
- When a command is new, read its help: `man <command>` or `<command> --help`.

3.1 What Linux is and where it is used

Linux is an open-source, Unix-like operating system. Strictly speaking, "Linux" is the **kernel** (written by Linus Torvalds in 1991); a complete system adds GNU tools, a shell, libraries and software. Together they are a **distribution (distro)**.

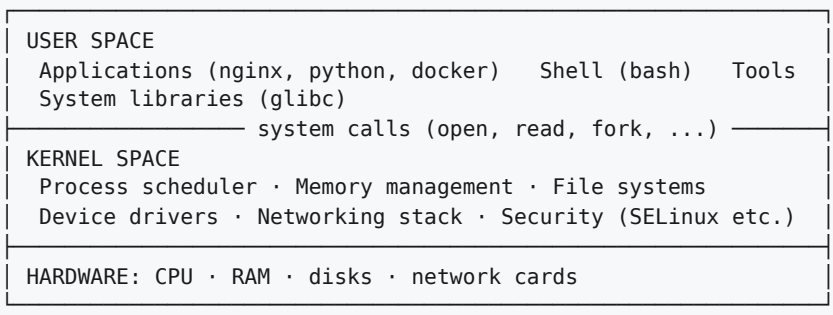
Where it runs: most web servers and cloud instances, almost every container image, Kubernetes nodes, network devices, Android phones, embedded systems/IoT, supercomputers. As a DevOps/Cloud engineer you will spend most of your working day on Linux servers, usually without a graphical interface.

Distribution families

Family	Distributions	Package format / tool	Where you meet it
Debian	Debian, **Ubuntu**, Linux Mint	`.deb` / `apt`	Very common on cloud servers, WSL, CI runners
Red Hat	**RHEL**, Rocky Linux, AlmaLinux, Fedora, CentOS Stream, **Amazon Linux**	`.rpm` / `dnf` (older: `yum`)	Enterprises, banks, AWS
SUSE	SLES, openSUSE	`.rpm` / `zypper`	European enterprises, SAP
Alpine	Alpine Linux	`apk`	Tiny container images
Arch	Arch Linux	`pacman`	Desktop enthusiasts

LTS (Long-Term Support) releases (e.g. Ubuntu 24.04 LTS, RHEL 9) get security updates for many years; servers should run LTS versions. Check the version: `cat /etc/os-release`, kernel: `uname -r`.

3.2 Linux architecture and boot process



- **Kernel:** the core. Only it talks to hardware. Programs ask it for services through **system calls**.
- **Shell:** a program that reads your commands and asks the kernel to run programs (`bash`, `zsh`, `sh`).
- **User space:** everything outside the kernel.

Boot sequence

```
Power on → Firmware (BIOS or UEFI) checks hardware, finds the boot disk
        → Bootloader (GRUB2) shows the menu, loads the kernel + initramfs
        → Kernel starts, detects hardware, mounts the root file system
        → systemd starts as PID 1 (the first process)
        → systemd starts services to reach the default target (multi-user / graphical)
        → login prompt / SSH available
```

Piece	Location / command
GRUB config	`/etc/default/grub`, apply with `sudo update-grub` (Ubuntu) or `grub2-mkconfig` (RHEL)
Kernel and initramfs	`/boot/vmlinuz-*`, `/boot/initrd.img-*` / `initramfs-*`
Default target	`systemctl get-default` (usually `multi-user.target` on servers)
Boot messages	`journalctl -b` (this boot), `dmesg` (kernel ring buffer)
Boot time analysis	`systemd-analyze`, `systemd-analyze blame`

Rescue options: `rescue.target` (single-user, minimal services) and `emergency.target` (root shell only) are chosen from GRUB by adding `systemd.unit=rescue.target` to the kernel line. On cloud instances use the **serial console** or attach the disk to another instance.

3.3 The shell and the command line

The prompt

```
thirupathi@web01:~/projects$
│          │     │         └ $ = normal user   (# = root)
│          │     └ current directory (~ = your home)
│          └ hostname
└ username
```

Command anatomy

```
ls  -l -a  --human-readable  /var/log
│   └─┬─┘ └────────┬───────┘  └──┬──┘
│   short options long option   argument (what to act on)
command
```

- Short options combine: `ls -la` = `ls -l -a`.
- Many commands accept `--` to mark the end of options (useful for file names starting with `-`).
- Commands are **case-sensitive**: `ls` works, `LS` does not.

Getting help

Command	What it gives
`man ls`	Full manual (`/word` search, `n` next, `q` quit). Sections: 1 commands, 5 file formats, 8 admin commands (`man 5 crontab`)
`ls --help`	Short usage summary

Command	What it gives
`apropos network` / `man -k network`	Search manual descriptions
`whatis ls`	One-line description
`type cd`	Is it a builtin, alias, function or file?
`which python3` / `command -v python3`	Path of the program that would run
`tldr tar`	Community examples (install `tldr`)

Keyboard shortcuts (bash)

Keys	Action
`Tab` / `Tab Tab`	Complete / show all completions
`↑` `↓`	Previous / next command
`Ctrl+R`	Search history (type part of a command)
`Ctrl+C`	Interrupt (stop) the running command
`Ctrl+D`	End of input / log out of the shell
`Ctrl+Z`	Suspend the running command (then `bg` or `fg`)
`Ctrl+L`	Clear the screen
`Ctrl+A` / `Ctrl+E`	Start / end of line
`Ctrl+U` / `Ctrl+K`	Delete to start / end of line
`Ctrl+W`	Delete previous word
`!!` / `sudo !!`	Repeat last command (with sudo)
`!$`	Last argument of the previous command
`history` / `!123`	Show history / run entry 123

Shells

`bash` (default on most distros), `sh` / `dash` (minimal, used for system scripts), `zsh` (macOS default, popular with plugins), `fish` (user-friendly). Current shell: `echo $SHELL`; change: `chsh -s /bin/zsh`.

3.4 The file system

Filesystem Hierarchy Standard (FHS)

```
/                   root of everything (one tree; no C:\ D:\ drives)
├── bin, sbin       essential commands (today usually links into /usr)
├── boot            kernel, initramfs, GRUB files
├── dev             device files (disks /dev/sda, /dev/nvme0n1, /dev/null)
├── etc             system-wide configuration (text files)
├── home            users' home directories (/home/thirupathi)
├── lib, lib64      shared libraries
├── media, mnt      mount points for removable/temporary file systems
├── opt             optional third-party software
├── proc            virtual: live process and kernel info (/proc/cpuinfo, /proc/<PID>)
├── root            home directory of the root user
├── run             runtime data since boot (PID files, sockets)
├── srv             data served by this system (web/ftp)
├── sys             virtual: devices and kernel settings
├── tmp             temporary files (often cleared on reboot)
├── usr             installed software (/usr/bin, /usr/lib, /usr/share, /usr/local)
└── var             variable data: logs (/var/log), caches, mail, web roots, databases
```

Most important for DevOps: `/etc` (config), `/var/log` (logs), `/home` (users), `/tmp`, `/opt` (apps), `/var/lib` (data of services like Docker, databases), `/proc` and `/sys` (live system info).

Everything is a file

Type (first character in `ls -l`)	Meaning
`-`	Regular file
`d`	Directory
`l`	Symbolic link
`b`	Block device (disk)
`c`	Character device (terminal, `/dev/null`)
`s`	Socket
`p`	Named pipe (FIFO)

Paths

Path	Meaning
`/etc/nginx/nginx.conf`	**Absolute**: starts at `/`
`logs/app.log`	**Relative**: from the current directory
`.` / `..`	Current / parent directory
`~` / `~alice`	Your home / alice's home
`-` (with `cd`)	Previous directory
`.bashrc`	Files starting with `.` are **hidden** (`ls -a` shows them)

File names are case-sensitive, can contain spaces (quote them: `"my file.txt"`), and have no required extension (`file` tells you the real type).

3.5 Working with files and directories

Command	Purpose	Useful options
`pwd`	Print working directory	
`ls`	List	`-l` long, `-a` hidden, `-h` human sizes, `-t` by time, `-S` by size, `-r` reverse, `-R` recursive, `-d` directory itself, `-i` inode numbers
`cd dir`	Change directory	`cd`, `cd ~`, `cd ..`, `cd -`
`mkdir dir`	Make directory	`-p` create parents, no error if exists
`touch file`	Create empty file / update timestamp	
`cp src dst`	Copy	`-r` recursive, `-p` keep permissions/times, `-a` archive (everything), `-i` ask before overwrite, `-v` verbose
`mv src dst`	Move or rename	`-i`, `-n` never overwrite
`rm file`	Remove	`-r` recursive, `-f` force, `-i` ask. **No recycle bin**
`rmdir dir`	Remove an empty directory	
`ln -s target link`	Symbolic link	
`file name`	Detect file type	
`stat name`	Size, permissions, inode, timestamps	
`tree -L 2`	Directory tree, 2 levels	

Safe deletion habits: run `ls` with the same pattern before `rm`; use `rm -i` for important files; never `rm -rf /` or `rm -rf $VAR/` with an unset variable (use `set -u` in scripts).

Links and inodes

An **inode** stores a file's metadata (owner, permissions, size, data block locations); the file name is just a directory entry pointing to an inode.

	Hard link (`ln a b`)	Symbolic link (`ln -s a b`)
Points to	The same inode	A path (name)
If original deleted	Data still exists	Link breaks ("dangling")
Across file systems	No	Yes
For directories	No	Yes
Typical use	Rare (backups, dedupe)	Very common: versions (`/opt/app/current → /opt/app/v2`), config switches

Wildcards (globbing) and expansion

Pattern	Matches	Example
`*`	Any characters	`ls *.log`

Pattern	Matches	Example
`?`	Exactly one character	`ls file?.txt`
`[abc]`, `[0-9]`	One character from a set/range	`ls app[12].conf`
`[!a]`	Any character except	`ls [!t]*`
`{a,b,c}`	Brace expansion (generates words)	`mkdir -p proj/{src,tests,docs}`
`{1..5}`	Sequence	`touch log{1..5}.txt`

The **shell** expands wildcards before the command runs. Quote patterns to stop expansion: `find . -name "*.log"`.

Quoting

Quote	Effect	Example
`'single'`	Everything literal	`echo '$HOME'` → `$HOME`
`"double"`	Variables and `$(...)` expand; spaces kept	`echo "Home: $HOME"`
`\`	Escape one character	`echo \$HOME`

3.6 Viewing files

Command	Use
`cat file`	Print whole file (small files)
`cat -n file`	With line numbers
`tac file`	Print in reverse line order
`less file`	Page through: `Space` / `b` page, `/text` search, `n` / `N` next/prev, `G` end, `g` start, `F` follow, `q` quit
`head -n 20 file`	First 20 lines
`tail -n 50 file`	Last 50 lines
`tail -f file`	Follow new lines (logs); `-F` also follows across log rotation
`wc -l file`	Count lines (`-w` words, `-c` bytes)
`diff -u old new`	Differences between files (unified format)
`cmp a b`	Byte-by-byte comparison
`md5sum` / `sha256sum file`	Checksums to verify files
`xxd file` / `hexdump -C file`	Hex view of binary files

3.7 Searching

find

```
find /var/log -name "*.log"                # by name (case-sensitive; -iname ignores case)
find . -type f -size +100M                 # files larger than 100 MB (-type d = directories)
find /tmp -type f -mtime +7                # modified more than 7 days ago (-mmin for minutes)
find /home -user alice                     # owned by alice
find / -perm -4000 -type f 2>/dev/null     # SUID files (security audit)
find . -maxdepth 1 -type f -empty          # empty files in this directory only
find /var/log -name "*.gz" -mtime +30 -delete              # delete old compressed logs
find . -name "*.sh" -exec chmod +x {} \;    # run a command on each result
find . -name "*.txt" -print0 | xargs -0 grep -l "TODO"     # safe with spaces in names
```

`locate name` searches a prebuilt database (fast; refresh with `sudo updatedb`).

grep

Option	Meaning
`grep "error" file`	Lines containing "error"
`-i`	Ignore case
`-r` / `-R`	Recursive through directories
`-n`	Show line numbers
`-v`	Invert: lines **not** matching
`-c`	Count matching lines
`-l`	Only file names with matches
`-w`	Whole words only
`-o`	Print only the matched part
`-E`	Extended regex, e.g. `grep -E "error|fail"` (error or fail)
`-A 3` / `-B 3` / `-C 3`	Lines after / before / around each match
`--include="*.py"`	Limit recursive search to file patterns

Regular expressions (regex) essentials

Pattern	Matches
`^start`	Line starting with "start"
`end$`	Line ending with "end"
`.`	Any single character
`*`	Previous item 0 or more times
`+`	1 or more (extended regex)
`?`	0 or 1 (extended regex)
`[0-9]`, `[a-z]`	Character ranges
`[^0-9]`	Not a digit
`{3}`, `{2,4}`	Exact / range repetitions (extended)

Pattern	Matches
`(a|b)`	Alternatives (extended)
`\b`	Word boundary

```
grep -E "^[0-9]{1,3}(\.[0-9]{1,3}){3}" access.log     # lines starting with an IPv4 address
grep -E " (4|5)[0-9]{2} " access.log                   # HTTP 4xx/5xx status codes
grep -rn --include="*.yaml" "image:" k8s/              # find image lines in YAML files
```

3.8 Text processing

Linux tools are small and combine through pipes. Learn these and you can analyze any log file.

Command	Purpose	Example
`cut`	Pick columns by delimiter or position	`cut -d: -f1,7 /etc/passwd` (user and shell)
`sort`	Sort lines	`-n` numeric, `-r` reverse, `-h` human sizes (`10K` , `2G`), `-k2` by column 2, `-t,` delimiter, `-u` unique
`uniq`	Collapse **adjacent** duplicates (sort first)	`-c` count, `-d` only duplicates
`tr`	Translate/delete characters	`tr 'a-z' 'A-Z'` , `tr -d '\r'` , `tr -s ' '` (squeeze spaces)
`paste` / `join` / `column -t`	Merge files side by side / join on a key / align columns	`column -t -s, data.csv`
`tee`	Write to a file **and** pass output on	`cmd | tee out.log`
`xargs`	Build commands from input	`cat ids.txt | xargs -n1 echo`

sed (stream editor)

```
sed 's/old/new/' file              # replace first match on each line (prints result)
sed 's/old/new/g' file             # replace all matches on each line
sed -i 's/8080/80/g' app.conf      # edit the file in place (-i.bak keeps a backup)
sed -n '10,20p' file               # print only lines 10-20
sed '/^#/d' file                   # delete comment lines
sed '/^$/d' file                   # delete empty lines
sed -i '3i new line' file          # insert a line before line 3
sed 's#/var/www#/srv/www#g' file   # any delimiter works (useful with paths)
```

awk (field processing)

awk splits each line into fields: `$1` , `$2` , ... (`$0` = whole line, `NF` = number of fields, `NR` = line number).

```
awk '{print $1}' access.log                          # first field
awk -F: '{print $1, $3}' /etc/passwd                 # custom separator
awk '$9 >= 500 {print $1, $7}' access.log            # condition: status code >= 500
awk '{sum += $10} END {print sum/1024/1024 " MB"}' access.log   # total bytes sent
awk 'NR==1 || /ERROR/' app.log                       # header line + error lines
awk '{count[$1]++} END {for (ip in count) print count[ip], ip}' access.log | sort -rn | head
printf "%-10s %5d\n" web 42                          # formatted output (also inside awk)
```

Log analysis one-liners

```
# Top 10 client IPs in an nginx access log
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head -10

# Count responses per status code
awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn

# Most requested URLs that returned 404
awk '$9 == 404 {print $7}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head

# Errors per hour in a syslog-style log
grep -i error /var/log/syslog | awk '{print $3}' | cut -d: -f1 | sort | uniq -c

# Failed SSH logins by source IP
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn
```

3.9 Text editors

nano (simplest)

`nano file` · shortcuts are shown at the bottom (`^` = Ctrl): `Ctrl+O` save, `Ctrl+X` exit, `Ctrl+W` search, `Ctrl+K` cut line, `Ctrl+U` paste, `Alt+U` undo.

vim (on every server, worth learning)

Vim has **modes**: **Normal** (move and run commands, default), **Insert** (type text), **Visual** (select), **Command-line** (`:` commands).

Keys (Normal mode)	Action
`i` / `a` / `o`	Insert before cursor / after cursor / new line below
`Esc`	Back to Normal mode
`h j k l`	Left, down, up, right
`w` / `b`	Next / previous word
`0` / `$`	Start / end of line
`gg` / `G` / `:42`	First line / last line / line 42
`x` / `dd` / `5dd`	Delete char / line / 5 lines
`yy` / `p`	Copy (yank) line / paste
`u` / `Ctrl+R`	Undo / redo
`/text` then `n`	Search forward, next match
`:%s/old/new/g`	Replace in the whole file
`v` / `V`	Visual select characters / lines
`:w` / `:q` / `:wq` / `:q!`	Save / quit / save and quit / quit without saving
`:set number`	Show line numbers

Survival rule: stuck in vim? Press `Esc`, type `:q!`, press Enter.

3.10 Redirection, pipes and exit codes

Every process has three standard **file descriptors**:

FD	Name	Default
0	stdin (input)	Keyboard
1	stdout (normal output)	Screen
2	stderr (errors)	Screen

Syntax	Meaning
`cmd > file`	stdout to file (overwrite)
`cmd >> file`	stdout appended
`cmd 2> err.log`	stderr to file
`cmd > all.log 2>&1`	stdout and stderr to the same file (order matters)
`cmd &> all.log`	Same, bash shortcut
`cmd > /dev/null 2>&1`	Discard everything
`cmd < file`	File as stdin
`cmd << EOF ... EOF`	Here-document (multi-line input)
`cmd <<< "text"`	Here-string
`cmd1 | cmd2`	Pipe stdout of cmd1 into stdin of cmd2
`cmd1 |& cmd2`	Pipe stdout and stderr
`$(cmd)`	Command substitution: replaced by the output
`<(cmd)`	Process substitution: output as a temporary file (`diff <(ls a) <(ls b)`)

Exit codes: every command returns a number: **0 = success**, anything else = failure. See it with `echo $?`.

Operator	Meaning
`cmd1 ; cmd2`	Run both, always
`cmd1 && cmd2`	Run cmd2 only if cmd1 succeeded
`cmd1 || cmd2`	Run cmd2 only if cmd1 failed
`cmd &`	Run in the background

3.11 Users and groups

Account files

```
/etc/passwd   alice:x:1001:1001:Alice Kumar:/home/alice:/bin/bash
              |     | |    |    |           |           └ login shell (/usr/sbin/nologin = no login)
              |     | |    |    |           └ home directory
              |     | |    |    └ comment (full name)
              |     | |    └ primary group ID (GID)
              |     | └ user ID (UID)
              |     └ x = password is in /etc/shadow
              └ username

/etc/shadow   alice:$6$...hash...:19700:0:90:7:::      (password hash + aging; root-only)
/etc/group    devops:x:1002:alice,bob                  (group name, GID, members)
```

UID range	Meaning
0	root (superuser)
1–999	System/service accounts (nginx, www-data, sshd)
1000+	Normal users

Commands

Command	Purpose
`id`, `id alice`	UID, GID, groups
`whoami`, `who`, `w`, `last`, `lastlog`	Current user / logged-in users / what they do / login history
`sudo useradd -m -s /bin/bash -c "Alice Kumar" alice`	Create user with home dir (`-m`), shell, comment
`sudo adduser alice`	Interactive friendly version (Debian/Ubuntu)
`sudo passwd alice`	Set password
`sudo usermod -aG docker alice`	**Append** to a supplementary group (without `-a` you replace all groups!)
`sudo usermod -L alice` / `-U`	Lock / unlock account
`sudo usermod -s /usr/sbin/nologin svc`	Disable interactive login for a service account
`sudo userdel -r alice`	Delete user and home
`sudo chage -l alice` / `chage -M 90 alice`	Show / set password aging
`sudo groupadd devops`, `groupdel`, `gpasswd -d alice devops`	Manage groups / remove member
`groups alice`	Group memberships
`/etc/skel/`	Template files copied into new home directories

Group changes apply at the **next login** (or run `newgrp docker`).

root, su and sudo

	`su - alice`	`sudo command`
Does	Switch to another user's shell (needs **their** password)	Run one command as root (needs **your** password)
Audit	Weak	Every command logged (`/var/log/auth.log` , `journalctl`)
Best practice	Avoid sharing root password	Grant specific users/groups specific rights

sudoers (edit only with `sudo visudo` , or files in `/etc/sudoers.d/`):

```
# who      where=(as whom)   which commands
%admins    ALL=(ALL:ALL)     ALL
deploy     ALL=(root)        NOPASSWD: /usr/bin/systemctl restart myapp
alice      ALL=(ALL)         /usr/bin/journalctl, /usr/bin/systemctl status *
```

`sudo -l` lists what you may run. `sudo -i` opens a root login shell (use sparingly).

3.12 Permissions in depth

```
-rwxr-x---  1  alice  devops  4096  Oct 17 10:00  deploy.sh
 |└┘└┘└┘     |      |
 | |  |  |      owner  group
 | |  |  └ others: ---
 | |  └ group:  r-x
 | └ owner (user): rwx
 └ type
```

Permission	On a file	On a directory
`r` (4)	Read contents	List names (`ls`)
`w` (2)	Modify contents	Create, delete, rename entries inside
`x` (1)	Execute as a program	Enter it (`cd`) and access files inside

Important: deleting a file depends on **write permission on the directory**, not on the file.

chmod

Numeric	Symbolic	Typical use
`755`	`rwxr-xr-x`	Scripts, programs, directories
`644`	`rw-r--r--`	Normal files, config
`600`	`rw-------`	Private keys, secrets
`700`	`rwx------`	Private directories (`~/.ssh`)
`640`	`rw-r-----`	Config readable by a service group
`775`	`rwxrwxr-x`	Shared team directories

```
chmod 640 app.conf
chmod u+x deploy.sh          # add execute for the owner (u=user, g=group, o=others, a=all)
chmod g-w,o= report.txt      # remove group write; others nothing
chmod -R 750 /srv/app        # recursive
chown alice:devops file      # owner and group
chown -R www-data: /var/www  # owner www-data, group = its primary group
chgrp devops file
```

umask

The **umask** removes permissions from new files/directories. Default files start at 666, directories at 777.

umask	New file	New directory
`022` (common default)	644	755
`027` (more secure)	640	750
`077` (private)	600	700

Show with `umask`; set in `~/.bashrc` or `/etc/login.defs`.

Special permission bits

Bit	Number	On a file	On a directory	Example
SUID	4000	Runs with the **owner's** privileges	—	`/usr/bin/passwd` (`-rwsr-xr-x`)
SGID	2000	Runs with the group's privileges	New files inherit the **directory's group**	Shared team folders: `chmod 2775 /srv/team`
Sticky	1000	—	Only the file owner can delete their files	`/tmp` (`drwxrwxrwt`)

Audit SUID files regularly: `find / -perm -4000 -type f 2>/dev/null`.

ACLs and attributes

- **ACLs** give permissions to extra users/groups: `setfacl -m u:bob:rx /srv/reports`, view with `getfacl /srv/reports` (a `+` appears in `ls -l`).
- **Attributes:** `sudo chattr +i file` makes a file immutable (even root can't change it until `chattr -i`); `lsattr` shows attributes.
- **Capabilities:** give a program one root power without full root, e.g. bind to port 80: `sudo setcap 'cap_net_bind_service=+ep' /usr/local/bin/app`; view with `getcap`.

Permission troubleshooting

`Permission denied` → check **every directory in the path** (`namei -l /srv/app/config/app.yaml` shows permissions along the path), the file mode, the owner/group, the process user (`ps -o user= -p <pid>`), ACLs, and SELinux/AppArmor (3.25).

3.13 Processes and jobs

A **process** is a running program. Each has a **PID**, a parent (**PPID**), an owner, priority, memory and open files. `systemd` is PID 1, the ancestor of all processes.

Process states

State (in `ps` / `top`)	Meaning
`R`	Running or ready to run
`S`	Sleeping (waiting for an event): normal for idle services
`D`	Uninterruptible sleep (waiting for disk/network I/O). Many `D` processes = storage problem
`T`	Stopped (Ctrl+Z or SIGSTOP)
`Z`	Zombie: finished but parent hasn't collected its exit status

Zombie: already dead, uses no CPU/RAM, only a process-table entry; fix the **parent** (restart it).
Orphan: parent died; adopted by systemd.

Viewing processes

Command	Use
`ps aux`	All processes (BSD style): USER, PID, %CPU, %MEM, VSZ, RSS, STAT, START, TIME, COMMAND
`ps -ef`	All processes (System V style) with PPID
`ps -ef --forest` / `pstree -p`	Parent/child tree
`ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%cpu | head`	Top CPU users, custom columns
`pgrep -a nginx`	Find PIDs by name
`top`	Live view: `P` sort CPU, `M` memory, `k` kill, `1` per-CPU, `q` quit
`htop`	Friendlier `top` (install it)
`/proc/<PID>/`	Live info: `cmdline`, `status`, `environ`, `fd/`
`lsof -p <PID>`	Files and sockets a process has open

RSS = physical memory used. **VSZ** = virtual memory reserved (often much larger; don't panic about it).

Signals

Signal	Number	Meaning	Send with
SIGTERM	15	Polite stop: process can clean up. **Default for `kill`**	`kill PID`
SIGKILL	9	Immediate kill by the kernel, no cleanup. **Last resort**	`kill -9 PID`
SIGINT	2	Interrupt (what Ctrl+C sends)	`kill -2 PID`
SIGHUP	1	Hang up; many daemons reload config on it	`kill -HUP PID`
SIGSTOP / SIGCONT	19 / 18	Pause / resume	`kill -STOP PID`

`pkill -f "python app.py"` (by command line), `killall nginx` (by name).

Foreground, background and jobs

Command	Use
`long_task &`	Start in the background
`Ctrl+Z` then `bg`	Suspend, then continue in the background
`jobs`, `fg %1`	List jobs / bring job 1 to the foreground
`nohup cmd > out.log 2>&1 &`	Keep running after logout
`disown`	Detach a job from the shell
`tmux` / `screen`	Persistent terminal sessions: survive SSH disconnects (`tmux new -s work`, detach `Ctrl+B d`, `tmux attach -t work`)

For real services use systemd (3.16), not `nohup`.

Priority

Nice values range from **-20** (highest priority) to **19** (lowest); default 0. Normal users can only lower priority. `nice -n 10 ./backup.sh` · `sudo renice -n 5 -p 1234` · for disk: `ionice -c3 cmd` (idle I/O class).

strace (advanced debugging)

`strace -p <PID>` or `strace -f -e trace=open,connect cmd` shows the system calls a program makes: which files it fails to open, which host it connects to. Powerful when logs say nothing.

3.14 System resources and performance

CPU

Command	What to look at
`nproc`, `lscpu`	Number of cores, CPU model
`uptime`	**Load average** for 1, 5, 15 minutes
`top` header `%Cpu(s)`	`us` user, `sy` kernel, `ni` niced, `id` idle, `wa` **I/O wait**, `st` stolen (by the hypervisor)
`mpstat -P ALL 1`	Per-core usage (package `sysstat`)
`pidstat 1`	Per-process CPU over time

Load average = average number of processes running **or waiting** (CPU or uninterruptible I/O). Compare with the core count: load 4 on a 4-core machine = fully busy; load 12 on 4 cores = overloaded. High load with low CPU usage usually means **I/O wait** (disk/network storage problem).

High `st` (steal) on a cloud VM = the host is overcommitted or a burstable instance is out of CPU credits.

Memory

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       2.1Gi       1.2Gi        45Mi       4.4Gi       5.3Gi
Swap:          2.0Gi          0B       2.0Gi
```

- **available** is the number that matters: memory that can be given to programs (includes reclaimable cache).

- **buff/cache** is used by the kernel to speed up disk access and is released when programs need it. "Free" being low is normal.
- **Swap** used heavily + slow system = not enough RAM (`vmstat 1`: watch `si` / `so` swap-in/out columns).
- **OOM killer:** when memory runs out, the kernel kills a process. Evidence: `dmesg -T | grep -i -E "killed process|out of memory"` or `journalctl -k | grep -i oom`.

Disk I/O

`iostat -xz 1` (`%util` near 100% and high `await` = saturated disk) · `iotop` (per-process I/O) · `vmstat 1` (`wa` column, `b` = blocked processes).

Network

`ss -s` (socket summary) · `ss -tan state established | wc -l` (number of connections) · `iftop` / `nload` (live bandwidth) · `sar -n DEV 1` · `ip -s link` (errors/drops).

Performance method (USE)

For every resource (CPU, memory, disk, network) check **U**tilization, **S**aturation (queues, waiting), **E**rrors. Then find **which process** is responsible, then **why** (logs, recent change, traffic).

Quick 60-second checklist	
`uptime`	Load trend
`dmesg -T | tail`	Kernel errors, OOM kills, disk errors
`vmstat 1 5`	Run queue, memory, swap, I/O wait
`mpstat -P ALL 1 3`	CPU balance per core
`pidstat 1 3`	Which processes use CPU
`iostat -xz 1 3`	Disk saturation
`free -h`	Memory
`sar -n DEV 1 3`	Network throughput
`top`	Overall view

3.15 Package management

Concepts: a **package** is software + metadata (version, dependencies, install scripts). **Repositories** are servers with packages and signed indexes. The package manager resolves **dependencies** and verifies **GPG signatures**.

Debian/Ubuntu: apt and dpkg

Command	Purpose
`sudo apt update`	Refresh package lists (do this first)
`sudo apt upgrade`	Upgrade installed packages
`sudo apt full-upgrade`	Upgrade, allowing removals/new dependencies
`sudo apt install nginx curl`	Install
`sudo apt install nginx=1.24.0-2ubuntu7`	Install a specific version
`sudo apt remove nginx` / `purge nginx`	Remove (purge also deletes config)

Command	Purpose
`sudo apt autoremove`	Remove unneeded dependencies
`apt search redis`, `apt show redis-server`	Search / details
`apt list --installed`, `apt list --upgradable`	Installed / upgradable
`apt-cache policy nginx`	Installed vs candidate versions and their repos
`sudo apt-mark hold nginx`	Prevent upgrades of a package
`dpkg -l | grep nginx`	Installed package list
`dpkg -L nginx`	Files installed by a package
`dpkg -S /usr/sbin/nginx`	Which package owns a file
`sudo dpkg -i package.deb`	Install a local .deb (then `sudo apt -f install` for dependencies)

Adding a third-party repository (modern, keyring-based):

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
```

Repositories are configured in `/etc/apt/sources.list` and `/etc/apt/sources.list.d/`.

RHEL/Amazon Linux/Rocky: dnf and rpm

Command	Purpose
`sudo dnf check-update` / `sudo dnf upgrade`	List / install updates
`sudo dnf install nginx`	Install
`sudo dnf remove nginx`	Remove
`dnf search`, `dnf info`, `dnf list installed`	Search / details / installed
`dnf provides /usr/bin/dig`	Which package provides a file
`dnf history`, `sudo dnf history undo <id>`	Transaction history / roll back
`dnf repolist`	Enabled repositories (`/etc/yum.repos.d/`)
`rpm -qa`, `rpm -qi nginx`, `rpm -ql nginx`, `rpm -qf /path`	Query installed packages, info, files, owner

Other ways software is installed

Method	Note
Snap / Flatpak	Self-contained packages (`snap install`)
Language package managers	`pip`, `npm`, `gem`, `go install` (use virtual environments / per-project installs)
Single binaries	Many DevOps tools (Terraform, kubectl, Helm) are one binary: download, verify checksum, put in `/usr/local/bin`

Method	Note
Build from source	`./configure && make && sudo make install` (rare today)
Containers	Increasingly, software is run as a container image instead of being installed

Security: install only from trusted, signed repositories; keep systems patched; on servers enable **unattended security upgrades** (`sudo apt install unattended-upgrades`) or `dnf-automatic`.

3.16 systemd and services

systemd is the init system (PID 1) and service manager on almost all modern distros. It starts services in parallel, restarts them on failure, manages logs (journald), timers, mounts and targets.

Unit types

Unit	Purpose
`.service`	A daemon or one-off task
`.socket`	Socket activation (start service on first connection)
`.timer`	Scheduled activation (cron alternative)
`.target`	Group of units / system state (`multi-user.target`)
`.mount` / `.automount`	File system mounts
`.path`	Trigger on file changes

Unit file locations: `/usr/lib/systemd/system/` or `/lib/systemd/system/` (from packages: **don't edit**) · `/etc/systemd/system/` (yours and overrides: **takes priority**).

systemctl

Command	Purpose
`systemctl status nginx`	State, PID, memory, last log lines
`sudo systemctl start / stop / restart nginx`	Control now
`sudo systemctl reload nginx`	Reload config without dropping connections (if supported)
`sudo systemctl enable --now nginx`	Start at boot **and** now
`sudo systemctl disable nginx`	Don't start at boot
`sudo systemctl mask nginx`	Make it impossible to start
`systemctl is-active nginx`, `is-enabled nginx`	Scriptable checks
`systemctl --failed`	Failed units
`systemctl list-units --type=service --state=running`	Running services
`systemctl list-unit-files`	Installed units and enabled state
`systemctl cat nginx`	Show the unit file in use (including overrides)
`sudo systemctl edit nginx`	Create an override (drop-in) without touching the package file

Command	Purpose
`sudo systemctl daemon-reload`	Reload unit files after editing
`systemctl list-dependencies multi-user.target`	Dependency tree

Writing a service unit

`/etc/systemd/system/shop-api.service`:

```
[Unit]
Description=Shop API
Documentation=https://github.com/<owner>/shop-api
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=shop
Group=shop
WorkingDirectory=/opt/shop-api
EnvironmentFile=/etc/shop-api/env
ExecStart=/opt/shop-api/.venv/bin/gunicorn -b 127.0.0.1:8000 app:app
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5
LimitNOFILE=65536
# Hardening
NoNewPrivileges=true
ProtectSystem=full
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Directive	Meaning
`After` / `Wants` / `Requires`	Start order / soft dependency / hard dependency
`Type`	`simple` (default, process stays in foreground), `forking` (daemonizes), `oneshot` (runs and exits), `notify` (tells systemd when ready)
`User`, `Group`	Run as an unprivileged account (create with `useradd -r -s /usr/sbin/nologin shop`)
`EnvironmentFile`	Variables from a file (permissions 600; keeps secrets out of the unit)
`Restart`	`on-failure`, `always`, `no`
`LimitNOFILE`	Max open files (raise for busy servers)
`WantedBy=multi-user.target`	`enable` links it into the normal boot

Then: `sudo systemctl daemon-reload && sudo systemctl enable --now shop-api && systemctl status shop-api`.

systemd timers (cron alternative)

`/etc/systemd/system/backup.service` (Type=oneshot running the script) + `/etc/systemd/system/backup.timer`:

```
[Unit]
Description=Nightly backup

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
RandomizedDelaySec=300

[Install]
WantedBy=timers.target
```

`sudo systemctl enable --now backup.timer` · `systemctl list-timers`. Advantages over cron: logs in the journal, `Persistent=true` runs missed jobs after downtime, dependencies, resource limits.

Targets

Target	Meaning
`multi-user.target`	Normal server mode (network, no GUI)
`graphical.target`	With desktop
`rescue.target`	Single-user maintenance
`emergency.target`	Minimal root shell

`systemctl get-default`, `sudo systemctl set-default multi-user.target`.

3.17 Logs

Where logs live

Source	Location
systemd journal (all services, kernel)	`journalctl` (binary; persistent if `/var/log/journal/` exists)
General system log	`/var/log/syslog` (Ubuntu) · `/var/log/messages` (RHEL)
Authentication, sudo, SSH	`/var/log/auth.log` (Ubuntu) · `/var/log/secure` (RHEL)
Kernel	`dmesg`, `/var/log/kern.log`
Package installs	`/var/log/apt/history.log` · `/var/log/dnf.log`
Applications	`/var/log/nginx/`, `/var/log/mysql/`, app-specific
Boot/cloud-init	`/var/log/cloud-init-output.log`

journalctl

Command	Purpose
`journalctl -u nginx`	Logs of one unit
`journalctl -u nginx -f`	Follow live
`journalctl -u nginx -n 100 --no-pager`	Last 100 lines
`journalctl --since "1 hour ago"` / `--since "2026-10-17 09:00" --until "10:00"`	Time window

Command	Purpose
`journalctl -p err -b`	Errors (priority ≤ err) since the current boot
`journalctl -b -1`	Previous boot (needs persistent journal)
`journalctl -k`	Kernel messages
`journalctl _PID=1234` / `_UID=1001`	By process / user
`journalctl -o json-pretty -u app`	Structured output
`journalctl --disk-usage`, `sudo journalctl --vacuum-size=500M`	Size / shrink

Log levels (syslog priorities): 0 emerg · 1 alert · 2 crit · 3 err · 4 warning · 5 notice · 6 info · 7 debug.

Log rotation

`logrotate` (run daily) rotates, compresses and deletes old logs. Config: `/etc/logrotate.conf` and `/etc/logrotate.d/`.

```
/var/log/shop-api/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 shop shop
    postrotate
        systemctl reload shop-api > /dev/null 2>&1 || true
    endscript
}
```

Test: `sudo logrotate -d /etc/logrotate.d/shop-api` (dry run) · `sudo logrotate -f ...` (force).

In production: ship logs off the server (CloudWatch agent, Fluent Bit, Vector, Promtail/Alloy → Loki, Elastic) so they survive server loss and can be searched centrally.

3.18 Storage, file systems and LVM

Concepts

```
Physical disk (/dev/sda, /dev/nvme0n1)
   └── Partition table (GPT or MBR)
         ├── Partition 1 (/dev/sda1)  → file system (ext4/xfs) → mounted at /
         └── Partition 2 (/dev/sda2)  → LVM PV → volume group → logical volumes → file systems
```

Term	Meaning
Block device	Disk or partition (`/dev/sda`, `/dev/nvme0n1p1`, `/dev/xvdf`)
Partition table	**GPT** (modern, large disks, many partitions) or **MBR** (legacy, 2 TB limit, 4 primary)
File system	How data is organized: **ext4** (Ubuntu default), **xfs** (RHEL/Amazon Linux default, can grow but not shrink), btrfs, vfat (EFI)
Mount	Attach a file system to a directory (mount point)

Term	Meaning
UUID	Unique ID of a file system: use it in `/etc/fstab` (device names can change)
Inode	Metadata record per file; a file system can run out of inodes before space

Inspecting storage

Command	Shows
`lsblk -f`	Disks, partitions, file systems, UUIDs, mount points
`df -hT`	Used/free space per mounted file system with type
`df -i`	Inode usage
`du -sh /var/*`	Size of each item (`--max-depth=1`)
`du -ah /var/log | sort -rh | head -20`	20 biggest files/dirs
`blkid`	UUIDs and types
`mount | column -t` , `findmnt`	Current mounts
`sudo fdisk -l`	Partition tables

Partition, format, mount (new disk)

```
sudo parted /dev/sdb --script mklabel gpt mkpart primary xfs 0% 100%
sudo mkfs.xfs /dev/sdb1                # or: sudo mkfs.ext4 /dev/sdb1   (ERASES data)
sudo mkdir -p /data
sudo mount /dev/sdb1 /data
sudo blkid /dev/sdb1                   # copy the UUID
echo 'UUID=<uuid>  /data  xfs  defaults,nofail  0  2' | sudo tee -a /etc/fstab
sudo umount /data && sudo mount -a     # test fstab BEFORE rebooting
```

/etc/fstab fields

```
UUID=1234-abcd   /data   xfs    defaults,nofail   0   2
|                |       |      |                 |   └ fsck order (0 skip, 1 root, 2 others)
|                |       |      |                 └ dump (0)
|                |       |      └ options (defaults, noatime, ro, nofail, _netdev for network FS)
|                |       └ file system type
|                └ mount point
└ device (UUID preferred)
```

A broken fstab line can stop the server from booting: always run `sudo mount -a` after editing and use `nofail` for non-critical disks.

LVM (Logical Volume Manager)

LVM adds a flexible layer: combine disks and resize volumes without repartitioning.

```
 /dev/sdb   /dev/sdc           ← PV  (physical volumes)
     └─────┬─────┘
        vg_data                ← VG  (volume group = pool of space)
     ┌─────┴─────┐
  lv_app     lv_logs           ← LV  (logical volumes = flexible "partitions")
  /srv/app   /var/log/app      ← file systems mounted
```

```
sudo pvcreate /dev/sdb /dev/sdc
sudo vgcreate vg_data /dev/sdb /dev/sdc
sudo lvcreate -n lv_app -L 20G vg_data
sudo mkfs.xfs /dev/vg_data/lv_app
sudo mount /dev/vg_data/lv_app /srv/app

# Grow later: add a disk, extend the VG, extend the LV and the file system in one step (-r)
sudo pvcreate /dev/sdd && sudo vgextend vg_data /dev/sdd
sudo lvextend -r -L +10G /dev/vg_data/lv_app

pvs; vgs; lvs                               # summaries
```

Snapshots: `lvcreate -s` creates point-in-time copies (useful before risky changes).

Growing a cloud disk (e.g. AWS EBS)

```
lsblk                                       # disk bigger than the partition?
sudo growpart /dev/nvme0n1 1                # grow partition 1
sudo xfs_growfs /                           # XFS (takes the mount point)
# ext4 instead: sudo resize2fs /dev/nvme0n1p1
df -h /
```

Swap

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
swapon --show; free -h
```

`vm.swappiness` (0–100, `sysctl vm.swappiness`) controls how eagerly the kernel swaps.

Other storage topics

Topic	Note
RAID (`mdadm`)	Software RAID 0 (speed), 1 (mirror), 5/6 (parity), 10
NFS client	`sudo mount -t nfs server:/export /mnt/share` (fstab option `_netdev`)
fsck	Check/repair **unmounted** file systems: `sudo fsck /dev/sdb1` (xfs: `xfs_repair`)
`noatime` mount option	Don't update access times: less disk writes

3.19 Archiving, compression and file transfer

Command	Purpose
`tar -czvf backup.tar.gz /etc/nginx`	Create gzip archive (c create, z gzip, v verbose, f file)
`tar -cJvf backup.tar.xz dir/`	xz compression (smaller, slower); `-j` = bzip2
`tar -tzvf backup.tar.gz`	List contents
`tar -xzvf backup.tar.gz -C /restore`	Extract into a directory
`tar --exclude='*.log' -czf app.tgz app/`	Exclude patterns
`gzip file` / `gunzip file.gz` / `zcat file.gz`	Compress / decompress / read compressed
`zip -r site.zip site/` / `unzip site.zip -d out/`	Zip format

Command	Purpose
`rsync -avz --delete src/ user@host:/dst/`	Sync (a archive, v verbose, z compress, delete extra files at destination)
`rsync -avzn ...`	Dry run first (`-n`)
`scp file user@host:/tmp/` / `scp -r dir host:`	Copy over SSH
`sftp user@host`	Interactive SSH file transfer
`curl -LO https://example.com/file.tgz` / `wget URL`	Download
`dd if=/dev/zero of=test bs=1M count=100`	Low-level copy (disk images, tests). Careful: `of=` a wrong disk destroys it

Trailing slash in rsync: `src/` copies the *contents* of src; `src` copies the directory itself.

3.20 Environment and shell configuration

Variables

Type	Example	Visible to child processes?
Shell variable	`NAME=web`	No
Environment variable	`export NAME=web`	Yes

Command	Purpose
`env` / `printenv`	Environment variables
`echo "$PATH"`	Directories searched for commands (left to right)
`export PATH="$HOME/bin:$PATH"`	Add a directory to PATH
`unset NAME`	Remove
`NAME=value command`	Set only for that one command
`set -o` / `shopt`	Shell options

Important variables: `PATH` , `HOME` , `USER` , `SHELL` , `PWD` , `LANG` / `LC_ALL` (locale), `EDITOR` , `HISTSIZE` , `PS1` (prompt), `TZ` .

Startup files

File	When it runs
`/etc/profile` , `/etc/profile.d/*.sh`	Login shells, all users
`~/.bash_profile` or `~/.profile`	Your login shells (SSH login)
`~/.bashrc`	Every interactive non-login shell (new terminal). Usually sourced from `.bash_profile`
`/etc/environment`	System-wide variables (simple `KEY=value` lines)

Reload after editing: `source ~/.bashrc` .

```
# ~/.bashrc additions
alias ll='ls -lah'
alias k='kubectl'
export EDITOR=vim
export HISTSIZE=10000
export HISTTIMEFORMAT='%F %T '
mkcd() { mkdir -p "$1" && cd "$1"; }
```

Secrets: never put passwords or keys in `.bashrc` or scripts committed to Git. Use a secrets manager, or a `chmod 600` env file loaded by the service.

3.21 Networking on Linux

Interfaces and addresses (`ip`)

Command	Purpose
`ip a` (`ip addr show`)	Interfaces and IPs (`lo` loopback, `eth0` / `ens5` / `enp0s3` network)
`ip -br a`	Brief view
`ip link set eth0 up/down`	Enable/disable interface
`sudo ip addr add 10.0.0.50/24 dev eth0`	Add an IP (temporary, lost at reboot)
`ip route` / `ip r`	Routing table (`default via 10.0.0.1` = gateway)
`ip route get 8.8.8.8`	Which route/interface is used for a destination
`sudo ip route add 10.20.0.0/16 via 10.0.0.254`	Add a static route (temporary)
`ip neigh`	ARP/neighbour table
`ip -s link`	Packet counters, errors, drops

`ifconfig`, `route`, `netstat`, `arp` are older (net-tools) equivalents you'll still see in docs.

Persistent network configuration

Ubuntu (netplan) `/etc/netplan/01-netcfg.yaml`, apply with `sudo netplan try` / `sudo netplan apply`:

```
network:
  version: 2
  ethernets:
    ens5:
      dhcp4: false
      addresses: [10.0.1.50/24]
      routes:
        - to: default
          via: 10.0.1.1
      nameservers:
        addresses: [10.0.0.2, 1.1.1.1]
```

RHEL family (NetworkManager): `nmcli con show`, `nmcli con mod ens5 ipv4.addresses 10.0.1.50/24 ipv4.gateway 10.0.1.1 ipv4.method manual`, `nmcli con up ens5`. On cloud instances, networking is normally DHCP-managed by the platform: don't hard-code IPs.

Name resolution

Item	Detail
`/etc/hosts`	Static name → IP entries, checked first (`10.0.1.20 db.internal`)
`/etc/resolv.conf`	DNS servers and search domains (often managed by systemd-resolved: `nameserver 127.0.0.53`)
`/etc/nsswitch.conf`	Order of lookups (`hosts: files dns`)
`resolvectl status`	Real upstream DNS servers with systemd-resolved
`hostnamectl set-hostname web01`	Set hostname
`getent hosts name`	Resolve exactly as applications do (honours /etc/hosts)

Diagnostic tools

Tool	Use
`ping -c 4 host`	Reachability and latency (ICMP may be blocked by firewalls)
`traceroute host` / `tracepath host` / `mtr host`	Path and where packets stop/slow down
`dig name` , `dig +short name` , `dig @8.8.8.8 name` , `dig name MX`	DNS queries (answer, TTL, server used)
`nslookup name` , `host name`	Simpler DNS lookups
`curl -v https://site`	Full HTTP(S) exchange: DNS, TCP, TLS, headers
`curl -I URL` / `curl -o /dev/null -s -w "%{http_code} %{time_total}\n" URL`	Headers only / status code and timing
`wget URL`	Download
`ss -tulpn`	Listening TCP/UDP ports with process (`t` TCP, `u` UDP, `l` listening, `p` process, `n` numeric)
`ss -tan`	All TCP connections and states (ESTAB, TIME-WAIT, SYN-SENT…)
`nc -zv host 443`	Test whether a TCP port is open
`nc -l 9000`	Quick listener for testing
`sudo tcpdump -i any -nn port 443`	Capture packets (`-w file.pcap` to save for Wireshark)
`openssl s_client -connect site:443 -servername site`	Inspect TLS certificate and handshake

`ss` output example: `LISTEN 0 511 0.0.0.0:80 0.0.0.0:* users:(("nginx",pid=812,fd=6))` → nginx listens on port 80 on all IPv4 addresses. `127.0.0.1:8000` would mean **local only**: not reachable from other machines.

Firewalls

Tool	Where	Examples
`ufw`	Ubuntu (simple front-end)	`sudo ufw default deny incoming`, `sudo ufw allow 22/tcp`, `sudo ufw allow from 10.0.0.0/16 to any port 5432`, `sudo ufw enable`, `sudo ufw status numbered`
`firewalld`	RHEL family	`sudo firewall-cmd --add-service=http --permanent`, `--add-port=8080/tcp --permanent`, `sudo firewall-cmd --reload`, `--list-all`
`nftables` / `iptables`	Underlying kernel firewall	`sudo nft list ruleset`, `sudo iptables -L -n -v` (Docker and Kubernetes write rules here)

Before enabling a firewall over SSH, allow port 22 first or you lock yourself out. In the cloud, security groups are the main firewall; host firewalls add defense in depth.

Kernel network settings (sysctl)

`sysctl net.ipv4.ip_forward` · enable routing: `sudo sysctl -w net.ipv4.ip_forward=1` · persist in `/etc/sysctl.d/99-custom.conf` · reload with `sudo sysctl --system`. Common tuning: `net.core.somaxconn`, `fs.file-max`, `net.ipv4.ip_local_port_range`.

3.22 SSH in depth

SSH (Secure Shell) gives encrypted remote login, command execution, file transfer and tunnels (TCP port 22). Server daemon: `sshd`; client: `ssh`.

Key-based authentication

```
Your laptop                                    Server
~/.ssh/id_ed25519      (PRIVATE, never share)
~/.ssh/id_ed25519.pub  ── copied to ──►  ~/.ssh/authorized_keys (one public key per line)
Login: server sends a challenge → your private key signs it → server verifies with the public key
```

```
ssh-keygen -t ed25519 -C "thirupathi@laptop"      # create key pair (set a passphrase)
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@host     # install the public key on the server
ssh user@host                                      # log in
ssh -i ~/.ssh/aws-lab.pem ec2-user@203.0.113.10   # use a specific key
ssh user@host 'df -h; uptime'                      # run remote commands
```

Required permissions (SSH refuses keys that are too open): `~/.ssh` → 700 · private key → 600 · `authorized_keys` → 600 · home directory not group/world-writable.

Client configuration `~/.ssh/config`

```
Host bastion
    HostName 203.0.113.10
    User ubuntu
    IdentityFile ~/.ssh/aws-lab.pem

Host app-*
    User ec2-user
    IdentityFile ~/.ssh/aws-lab.pem
    ProxyJump bastion

Host *
    ServerAliveInterval 60
    IdentitiesOnly yes
```

Now `ssh app-1` jumps through the bastion automatically (`ssh -J bastion user@10.0.11.5` does the same on the command line).

ssh-agent

Holds decrypted keys in memory so you type the passphrase once: `eval "$(ssh-agent -s)"` · `ssh-add ~/.ssh/id_ed25519` · `ssh-add -l`. Agent forwarding (`-A`) is convenient but risky on untrusted servers; prefer `ProxyJump`.

Port forwarding (tunnels)

Type	Command	Use
Local	`ssh -L 5433:db.internal:5432 bastion`	Reach a private database at `localhost:5433` through the bastion
Remote	`ssh -R 8080:localhost:3000 server`	Expose your local port 3000 on the server's port 8080
Dynamic (SOCKS)	`ssh -D 1080 bastion`	Browser proxy through the server

Server hardening (`/etc/ssh/sshd_config` or `/etc/ssh/sshd_config.d/*.conf`)

```
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowGroups ssh-users
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
```

Validate before restarting: `sudo sshd -t` → then `sudo systemctl reload ssh` (Ubuntu) / `sshd` (RHEL). **Keep your current session open** and test a new login before closing it.

Extra protection: firewall/security group allowing 22 only from known IPs or a VPN, `fail2ban` (bans IPs after failed attempts), or no SSH at all (AWS SSM Session Manager).

known_hosts and fingerprints

The first connection asks you to trust the server's **host key** (saved in `~/.ssh/known_hosts`). `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED` means the key changed: expected if the server was rebuilt (remove the old entry with `ssh-keygen -R host`), dangerous otherwise (possible man-in-the-middle).

SSH troubleshooting

Symptom	Check
`Connection timed out`	Network path/firewall/security group/route; server running; correct IP
`Connection refused`	Host reachable but nothing on port 22: sshd stopped or different port
`Permission denied (publickey)`	Wrong user/key; key not in `authorized_keys`; permissions too open; `PubkeyAuthentication`
`Host key verification failed`	Changed host key (see above)
Slow login	Reverse DNS lookups (`UseDNS no`), GSSAPI (`GSSAPIAuthentication no`)
Debug	`ssh -vvv user@host` on the client; `sudo journalctl -u ssh -f` on the server

3.23 Scheduling: cron, at and timers

cron syntax

```
┌──────────── minute (0-59)
│ ┌────────── hour (0-23)
│ │ ┌──────── day of month (1-31)
│ │ │ ┌────── month (1-12)
│ │ │ │ ┌──── day of week (0-7, Sunday = 0 or 7)
│ │ │ │ │
* * * * *  command
```

Expression	Runs
`*/5 * * * *`	Every 5 minutes
`0 2 * * *`	Daily at 02:00
`30 9 * * 1-5`	09:30 Monday–Friday
`0 0 1 * *`	Midnight on the 1st of each month
`0 */6 * * *`	Every 6 hours
`@reboot` / `@daily` / `@weekly`	Special strings

Where	Notes
`crontab -e` / `crontab -l` / `crontab -r`	Per-user crontab (edit / list / remove all!)
`/etc/crontab`, `/etc/cron.d/*`	System crontabs: have an extra **user** field: `0 2 * * * root /opt/backup.sh`
`/etc/cron.daily/` etc.	Scripts run daily/weekly/monthly

Cron gotchas: minimal environment (set `PATH` in the crontab or use full paths), no terminal, `%` must be escaped as `\%`, output is lost unless redirected: `0 2 * * * /opt/backup.sh >> /var/log/backup.log 2>&1`. Check runs in `grep CRON /var/log/syslog` or `journalctl -u cron`.

`at`: one-off jobs: `echo "systemctl restart app" | at 23:00` · `atq` · `atrm`. Prefer **systemd timers** for important jobs (3.16).

3.24 Time synchronization

Correct time matters for logs, TLS certificates, Kerberos, distributed databases and AWS API signatures. Modern distros use **chrony** or **systemd-timesyncd**.

`timedatectl` (time zone, NTP sync status) · `sudo timedatectl set-timezone Asia/Kolkata` · `chronyc tracking` / `chronyc sources -v`. On AWS, the Amazon Time Sync Service is available at `169.254.169.123`. Servers usually run in **UTC**.

3.25 Linux security and hardening

Hardening checklist (servers)

Area	Actions
Updates	Patch regularly; automatic security updates (`unattended-upgrades` / `dnf-automatic`); reboot for kernel updates (or live patching)
Accounts	Named accounts (no shared root), least privilege via sudo, lock unused accounts, service accounts with `nologin` shell, strong password policy (PAM `pam_pwquality` , lockout with `faillock`)
SSH	Keys only, no root login, restricted source IPs, fail2ban, or replace with SSM/VPN
Network	Firewall default-deny inbound; close unused ports (`ss -tulpn`); disable unused services (`systemctl disable --now`)
Files	Correct permissions on secrets (600), audit SUID/SGID and world-writable files, `/tmp` mounted `noexec` where possible
Mandatory access control	Keep **SELinux** (RHEL) or **AppArmor** (Ubuntu) **enforcing**
Logging/audit	Persistent journal, central log shipping, `auditd` rules for sensitive files
Integrity	File integrity monitoring (AIDE), malware/rootkit checks
Baselines	CIS Benchmarks; scan with **Lynis** (`sudo lynis audit system`) or OpenSCAP

SELinux (RHEL, Rocky, Amazon Linux)

Labels every file and process with a **context**; policy decides which process types may access which file types, even if normal permissions allow it.

Command	Purpose
`getenforce` / `sestatus`	Mode: **Enforcing**, Permissive (log only), Disabled
`sudo setenforce 0`	Temporarily permissive (troubleshooting only)
`ls -Z /var/www/html`	File contexts (e.g. `httpd_sys_content_t`)
`ps -eZ | grep nginx`	Process contexts
`sudo restorecon -Rv /var/www/html`	Reset contexts to policy defaults (after moving files)
`sudo semanage fcontext -a -t httpd_sys_content_t "/srv/site(/.*)?"`	Define context for a custom path, then `restorecon`
`sudo semanage port -a -t http_port_t -p tcp 8081`	Allow a service on a non-standard port

Command	Purpose
`getsebool -a | grep httpd`, `sudo setsebool -P httpd_can_network_connect on`	Booleans (e.g. let nginx proxy to backends)
`sudo ausearch -m avc -ts recent`, `audit2why`	Read denials and why they happened

Never "fix" problems by disabling SELinux permanently: find the right context/boolean.

AppArmor (Ubuntu)

Per-program profiles (`/etc/apparmor.d/`). `sudo aa-status` shows loaded profiles; denials appear in `journalctl -k | grep apparmor`. Modes: enforce / complain (`aa-complain`).

auditd

Kernel-level auditing: `sudo auditctl -w /etc/passwd -p wa -k identity` watches writes/attribute changes; search with `ausearch -k identity`. Rules persist in `/etc/audit/rules.d/`.

3.26 Hands-on lab: nginx web server and reverse proxy

```
# 1) Install and start
sudo apt update && sudo apt install -y nginx          # RHEL: sudo dnf install -y nginx
sudo systemctl enable --now nginx
curl -I http://localhost                               # expect HTTP/1.1 200 OK

# 2) Inspect
ss -tulpn | grep :80
ls /etc/nginx/        # nginx.conf, sites-available/ + sites-enabled/ (Ubuntu), conf.d/
tail -f /var/log/nginx/access.log /var/log/nginx/error.log
```

Site config `/etc/nginx/sites-available/shop` (Ubuntu; on RHEL use `/etc/nginx/conf.d/shop.conf`):

```
upstream shop_backend {
    server 127.0.0.1:8001;
    server 127.0.0.1:8002;
}

server {
    listen 80;
    server_name shop.example.com;

    location /static/ {
        root /srv/shop;                    # serves /srv/shop/static/...
        expires 7d;
    }

    location / {
        proxy_pass http://shop_backend;     # round-robin load balancing
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    access_log /var/log/nginx/shop.access.log;
    error_log  /var/log/nginx/shop.error.log;
}
```

```
sudo ln -s /etc/nginx/sites-available/shop /etc/nginx/sites-enabled/shop
sudo nginx -t                        # ALWAYS test the configuration
sudo systemctl reload nginx

# Two test backends
mkdir -p /tmp/b1 /tmp/b2
echo backend-1 > /tmp/b1/index.html; echo backend-2 > /tmp/b2/index.html
(cd /tmp/b1 && python3 -m http.server 8001 &) ; (cd /tmp/b2 && python3 -m http.server 8002 &)
for i in 1 2 3 4; do curl -s -H "Host: shop.example.com" http://localhost/; done   # alternates
```

HTTPS: obtain a free certificate with Let's Encrypt (`sudo certbot --nginx -d shop.example.com`), or terminate TLS at a cloud load balancer.

Common nginx errors: `502 Bad Gateway` (backend down/wrong port/SELinux blocking `httpd_can_network_connect`), `504 Gateway Timeout` (backend slow), `403 Forbidden` (file permissions/SELinux context, missing index), `address already in use` (another process on port 80: `ss -tulpn`).

3.27 Troubleshooting scenarios

Method: 1) define the symptom exactly · 2) what changed? · 3) check status → logs → resources → network → configuration/permissions · 4) one change at a time · 5) verify and document.

Server is slow / high CPU

```
uptime                                  # load vs core count (nproc)
top                                     # press P: which process? us vs sy vs wa vs st?
ps -eo pid,user,%cpu,%mem,cmd --sort=-%cpu | head
pidstat 1 5                             # is it constant or spiky?
```

Causes: runaway loop/bug, traffic spike, cron job, crypto-miner (unknown process!), too many workers. Fix: restart/scale/limit (`systemd` `CPUQuota=`), investigate the code, block abusive traffic.

Out of memory / OOM kills

```
free -h; vmstat 1 5                     # available memory, swap activity (si/so)
ps -eo pid,user,%mem,rss,cmd --sort=-%mem | head
dmesg -T | grep -i -E "out of memory|killed process"
```

Causes: memory leak, too many workers/connections, JVM heap too large, cache without limits. Fix: restart the leaking service, set limits (`MemoryMax=` in the unit), right-size, add swap as a buffer, fix the leak.

Disk full

```
df -h                                   # which file system?
df -i                                   # inodes full? (millions of tiny files)
sudo du -xh / --max-depth=1 2>/dev/null | sort -rh | head
sudo du -xh /var --max-depth=2 2>/dev/null | sort -rh | head
sudo lsof +L1                           # deleted files still held open (space not freed)
```

Typical culprits: `/var/log` (logs without rotation), journal (`journalctl --vacuum-size=500M`), Docker (`docker system df` , `docker system prune`), core dumps, old kernels, application caches, backups written locally. If a deleted log is still open, **restart or reload the process** to release the space.

High I/O wait

`iostat -xz 1` (device `%util`, `await`), `iotop -o` (which process). Causes: heavy logging, database without enough IOPS, swapping, backups during peak hours. Fix: faster disk (gp3 IOPS), move logs, schedule heavy jobs, add RAM.

Service won't start

```
systemctl status app                      # exit code, last log lines
journalctl -u app -n 100 --no-pager       # full error
sudo -u appuser /opt/app/start.sh         # run it manually as the service user
ss -tulpn | grep :8000                    # port already in use?
namei -l /opt/app/config.yaml             # permissions along the path
```

Common causes: syntax error in config (`nginx -t`, `sshd -t`, `apachectl configtest`), port in use, missing environment variable/secret, wrong file ownership, SELinux denial, dependency (database) unavailable.

Cannot connect to a service on the server

Server side: is it running? listening on the right **address** (`0.0.0.0` vs `127.0.0.1`) and port (`ss -tulpn`)? local test (`curl localhost:port`)? host firewall (`ufw status`, `firewall-cmd --list-all`)? Then network side: security group/NACL, routing, DNS.

DNS not resolving

```
getent hosts api.example.com              # as applications see it
dig api.example.com                       # status NOERROR / NXDOMAIN / SERVFAIL, answer, TTL
dig @8.8.8.8 api.example.com              # compare with a public resolver
resolvectl status; cat /etc/resolv.conf   # which resolver is used?
grep api /etc/hosts                       # stale override?
```

"Too many open files"

`ulimit -n` (shell), `cat /proc/<PID>/limits` (process), `ls /proc/<PID>/fd | wc -l` (current usage). Raise with `LimitNOFILE=` in the systemd unit or `/etc/security/limits.conf` for login sessions; also check for file/socket leaks.

Time is wrong

`timedatectl` (NTP synchronized?), `chronyc tracking`; TLS errors "certificate not yet valid" often come from clock drift.

3.28 Command reference by task

Task	Commands
Where am I / what's here	`pwd`, `ls -lah`, `tree -L 2`
Find files	`find`, `locate`, `which`, `type`
Read files	`cat`, `less`, `head`, `tail -f`, `wc -l`
Search text	`grep -rni`, `awk`, `sed -n`, `cut`, `sort`, `uniq -c`
Edit	`nano`, `vim`, `sed -i`
Users	`id`, `useradd -m`, `usermod -aG`, `passwd`, `sudo -l`, `visudo`

Task	Commands
Permissions	`chmod`, `chown`, `umask`, `getfacl/setfacl`, `namei -l`
Processes	`ps aux`, `top`/`htop`, `pgrep`, `kill`, `nice`, `lsof -p`, `strace -p`
Services	`systemctl status/start/enable/restart/daemon-reload`, `journalctl -u -f`
Packages	`apt update/install/remove`, `dnf install/upgrade`, `dpkg -L`, `rpm -qf`
Resources	`uptime`, `free -h`, `vmstat 1`, `iostat -xz 1`, `df -h`, `du -sh`
Storage	`lsblk -f`, `blkid`, `mkfs`, `mount`, `/etc/fstab`, `pvs/vgs/lvs`, `growpart`, `xfs_growfs/resize2fs`
Network	`ip a`, `ip r`, `ss -tulpn`, `ping`, `mtr`, `dig`, `curl -v`, `nc -zv`, `tcpdump`
Remote	`ssh`, `ssh-keygen`, `ssh-copy-id`, `scp`, `rsync -avz`
Archive	`tar -czvf`, `tar -xzvf -C`, `gzip`, `zip`
Schedule	`crontab -e`, `systemctl list-timers`, `at`
Logs	`journalctl`, `/var/log/*`, `dmesg -T`, `logrotate -d`
Security	`ufw`, `firewall-cmd`, `getenforce`, `ausearch`, `lynis`, `find -perm -4000`
System info	`uname -a`, `cat /etc/os-release`, `hostnamectl`, `lscpu`, `lsblk`, `timedatectl`

3.29 Interview questions and answers

#	Question	Answer
1	What is the Linux kernel?	The core of the OS that manages CPU scheduling, memory, devices, file systems and networking; programs use it through system calls
2	Describe the Linux boot process	Firmware (BIOS/UEFI) → GRUB → kernel + initramfs → systemd (PID 1) → default target → services and login
3	Difference between a process and a thread?	A process has its own memory space; threads share the memory of their process
4	What is a zombie process?	A finished process whose parent hasn't read its exit status; fix by restarting/fixing the parent
5	`kill` vs `kill -9`?	SIGTERM lets the process clean up; SIGKILL kills immediately without cleanup (last resort)
6	What does load average mean?	Average number of runnable + uninterruptible processes over 1/5/15 minutes; compare to core count
7	How do you find which process uses a port?	`ss -tulpn | grep :8080` or `lsof -i :8080`
8	Explain `chmod 755` and `644`	755: owner rwx, group/others r-x (scripts/dirs). 644: owner rw, others read (files)
9	What are SUID, SGID and sticky bit?	SUID runs as file owner; SGID runs as group / dirs pass group to new files; sticky lets only owners delete files in a shared dir (`/tmp`)
10	Hard link vs soft link?	Hard link = another name for the same inode (same FS, no dirs); symlink = pointer to a path, can break, crosses file systems
11	What is an inode? Can a disk be full with free space?	Metadata record per file; yes, when inodes run out (`df -i`)

#	Question	Answer
12	Disk is full: what do you do?	`df -h` → `du` to find big dirs → logs/journal/docker/old files → check deleted-open files (`lsof +L1`) → add rotation/monitoring
13	`df` shows full but `du` shows less. Why?	Deleted files still held open by a process; restart/reload it
14	How do you make a mount permanent?	Add it to `/etc/fstab` with the UUID, then test with `mount -a`
15	What is LVM and why use it?	Abstraction over disks (PV → VG → LV) to resize/extend volumes and add disks without downtime
16	What does `free -h` "available" mean?	Memory that can be given to applications, including reclaimable cache
17	What is the OOM killer?	Kernel mechanism that kills a process when memory is exhausted; seen in `dmesg` /journal
18	`systemctl enable` vs `start` ?	`start` runs it now; `enable` starts it at boot (`enable --now` does both)
19	How do you create a custom service?	Unit file in `/etc/systemd/system/` with [Unit]/[Service]/[Install], `daemon-reload` , `enable --now`
20	Where do you look when a service fails?	`systemctl status` , `journalctl -u name` , app logs, config test commands
21	What does `2>&1` do?	Redirects stderr to wherever stdout goes
22	Difference between `>` and `>>` ?	Overwrite vs append
23	How do you schedule a job every 5 minutes?	Cron `*/5 * * * * /path/script.sh >> /var/log/x.log 2>&1` or a systemd timer
24	Why do cron jobs work manually but fail in cron?	Different environment: PATH, working directory, no TTY, permissions; use full paths and log output
25	`su` vs `sudo` ?	`su` switches user with the target's password; `sudo` runs commands with your password, per-command rights and auditing
26	How do you give a user admin rights?	Add to `sudo` / `wheel` group or a sudoers.d rule (via `visudo`)
27	How do you harden SSH?	Keys only, no root login, restricted IPs/groups, fail2ban, non-default exposure via VPN/SSM, keep updated
28	SSH says permission denied (publickey)	Wrong user/key, key not in authorized_keys, wrong permissions on ~/.ssh files
29	How do you check open ports and connections?	`ss -tulpn` , `ss -tan` , `lsof -i`
30	How do you troubleshoot DNS on Linux?	`getent hosts` , `dig` (and `dig @8.8.8.8`), `/etc/resolv.conf` , `resolvectl status` , `/etc/hosts`
31	What is `/proc` ?	Virtual file system exposing kernel and process information (e.g. `/proc/cpuinfo` , `/proc/<pid>/status`)
32	What is SELinux and what do you do if it blocks an app?	Mandatory access control; check `ausearch` / `audit2why` , fix contexts (`restorecon` / `semanage`) or booleans, don't disable it
33	How do you find files larger than 1 GB?	`find / -type f -size +1G 2>/dev/null`

#	Question	Answer
34	How do you see the last 100 lines of a log and follow it?	`tail -n 100 -f file` or `journalctl -u name -n 100 -f`
35	Explain `ps aux` columns RSS and VSZ	RSS = physical memory in use; VSZ = virtual memory reserved
36	How do you check the Linux distribution and kernel version?	`cat /etc/os-release`, `uname -r`
37	What is swap and is using it bad?	Disk used as overflow memory; light use is fine, heavy swapping means not enough RAM
38	How do you change the hostname permanently?	`hostnamectl set-hostname name` (and update `/etc/hosts` if needed)
39	What is `umask 022`?	Removes write for group/others: new files 644, new directories 755
40	A website on the server is down: your steps?	Service status → logs → `ss` listening → local `curl` → firewall → resources (disk/memory) → recent changes → fix and verify

3.30 Self-check: I can…

- [ ] Explain kernel vs shell vs distribution and describe the boot process.
- [ ] Navigate the file system confidently and explain what `/etc`, `/var`, `/proc`, `/home`, `/opt` contain.
- [ ] Create, copy, move, link, find and safely delete files; use wildcards and quoting correctly.
- [ ] Search and analyze logs with `grep`, `awk`, `sed`, `sort`, `uniq`, `cut` and pipes.
- [ ] Edit files in vim and nano.
- [ ] Use redirection (`>`, `>>`, `2>&1`), pipes and exit codes.
- [ ] Create users and groups, configure sudo safely, and explain `/etc/passwd` and `/etc/shadow`.
- [ ] Set permissions numerically and symbolically; explain umask, SUID/SGID/sticky, ACLs.
- [ ] Inspect and control processes; send the right signals; use jobs and tmux.
- [ ] Diagnose CPU, memory, disk I/O and network usage with standard tools.
- [ ] Install, update, pin and remove packages with apt and dnf; add a signed repository.
- [ ] Write a systemd service and timer; read logs with journalctl; configure logrotate.
- [ ] Partition, format, mount (fstab), extend with LVM and grow a cloud disk.
- [ ] Archive, compress and transfer files with tar, rsync and scp.
- [ ] Configure environment variables, PATH, aliases and shell startup files.
- [ ] Configure and troubleshoot networking: IPs, routes, DNS, ports, firewalls.
- [ ] Use SSH keys, config files, jump hosts, tunnels; harden sshd.
- [ ] Schedule jobs with cron and systemd timers.
- [ ] Harden a server and work with SELinux/AppArmor instead of disabling them.
- [ ] Set up nginx as a web server and reverse proxy.
- [ ] Troubleshoot high CPU, OOM, full disk, failed services, connectivity and DNS step by step.

REMEMBER

- Permissions: **r = 4, w = 2, x = 1** → 7 = rwx, 6 = rw-, 5 = r-x, 4 = r--. Private keys: **600**.
- Cron fields in order: **minute hour day month weekday** ("**M**y **H**amster **D**oesn't **M**ind **W**aiting").
- Service broken? **status → logs → resources → network → config**.
- Linux has no recycle bin: double-check before `rm -rf`.